

RBR - Room Bot Rumble - Team19

RBR - Room Bot Rumble - Team19 (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *RBR* is a bot-builder auto-battler: the player creates a battle cleaning bot by allocating points to its **statistics and behaviour** and attaching **gadgets/weapons**, then the bots fight until the timer runs out or a bot is destroyed — the higher score wins. It features an **asynchronous PVP** system with a points-based **rating/ranking**. Named technical features: async PVP with a points rating; modular bot customisation (gadgets/weapons); customisations with unique behaviours; stat customisation affecting behaviour; and a **paint-trail render** as the bot moves. Load-bearing C++: a 2D renderer (multiple objects, primitive shapes, images, text), a sound system, an object-based system for AIs/projectiles/components, a game loop updating & rendering objects/projectiles/particles, image-slicing for sprite-sheet animation, a collision system for combat, and a particle system for VFX. Tools: a **generic Finite State Machine** for bot AI, an **Entity-Component System** for modular objects, an **animation system** (sprite-sheet slicing + playback), a **UI system** (parent-child anchors), and a **module maker** to create new bot attachments. Design features: the bot-customisation system (weapon/gadget types); player-adjusted stats that also affect behaviour; timer/score-based combat; and an **auto-battler** where the player watches their built bot fight in a single world map.

References:

- [GLADIABOTS - AI Combat Arena on Steam](#) (build and customise a robot crew, construct their AI/behaviour, send them to the arena and watch it play out — with **asynchronous multiplayer** and ranked matches; the closest build-a-bot-set-its-behaviour + async-PvP precedent)

- [Super Auto Pets on Steam](#) (an **auto-battler** where you build a loadout and battle other players **asynchronously** — Arena mode with no timers, working toward wins/rating — the async auto-battler + rating precedent)
- [Robocraft — Wikipedia](#) (build a combat robot by attaching **modular parts and weapons** — lasers, launchers, rail guns, melee — then send it into battle; the modular gadget/weapon bot-building precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	RBR - Room Bot Rumble
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (device: Pixel 9a) and/or Web browser on mobile — an auto-battler + builder is touch-friendly on either; the Native PC build is the development/tools vehicle (the FSM/ECS tooling + the module maker). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts, submitted bot builds, the async-PVP match resolution and the points rating/ranking.

Front end: A 2D renderer (objects, primitives, images, text, sprite-sheet animation) of the bots, gadgets/weapons, projectiles, the paint trail and the world map, plus the anchored UI (the bot-builder screen, stat allocation, combat HUD, leaderboard). **Languages:** Web — HTML/JS/CSS + C++ → WASM (Emscripten, SDL2/3, OpenGL ES); Native — Kotlin Native Android UI + C++ via JNI (Android Studio); OpenGL ES in a GLSurfaceView where realtime 2D is needed (no SDL on native).

Back end: The real-time C++ combat simulation loop (FSM bot AI over the ECS, projectiles, collision, particles/paint-trail, timer/score) + the async-PVP service that resolves submitted bot builds and updates the points rating.

Languages: C++ (engine + sim); Firebase-backed services + the Linux match/rating host on the Extended tier.

Backend scope: Local now, server-ready (T2) — the accounts / bot-build submission / match-resolution / rating seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — it drives the FSM bot AI (behaviour derived from stats + attached gadgets), moves/collides projectiles, updates particles + the paint trail, and resolves the timer/score win condition; the async-PVP match resolution runs the same deterministic sim.
- *Data-driven:* bots, stat allocations, gadgets/weapons (each with unique behaviours), FSM behaviour definitions and world maps are all data via the ECS — a new module or bot is a data change (authored in the module maker).
- *2D:* the renderer (objects/primitives/images/text/sprite-sheets), the paint-trail, projectiles, particles and the anchored UI (custom-rendered). *3D:* not needed. *Networking:* PVP is **asynchronous** (submit build → server resolves → rating updates), so it's async client ↔ server (Application Communication ✓) and a real-time Networking System is optional (●).
- *Custom tool:* the **module maker** (author new gadgets/weapons/attachments + their behaviours and stats as data) — plus the FSM/ECS/animation/UI systems as reusable engine tooling.
- *External DB:* accounts, submitted bot builds, match results and the rating/ranking. *Auth* »: accounts for PVP/rating. *Analytics* »: optional — build/win-rate stats for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in `milestone-goals-pool-per-category.pdf` (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the combat loop + the generic FSM + the UI system.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a real-time C++ loop updating/rendering objects, projectiles and particles, a generic start/update/exit FSM for bot AI, a sound system, and an anchored parent-child UI system. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the bot AI (M1G8) and combat run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the ECS (modular bots/gadgets/components).

Tech & dependencies

- **Tech:** An entity-component system in C++ (transform, collision, stats, behaviour, weapon/gadget components) with bots, gadgets/weapons and their behaviours/stats in hot-reloadable JSON — a new module is a data change; low coupling between systems.
- **Prerequisites:** G1 (loop & system update order).
- **Feeds into:** M2G2 (the module maker), M1G8 (behaviour/stat data), M2G13 (build/stat economy).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui overlays (FSM state, stats, pool usage), Tracy profiling of a full fight with many projectiles + particles + paint-trail, Catch2 tests for the deterministic combat sim (same build → same result, needed for async PVP) and data loading, on a GitHub Actions matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the deterministic async-PVP resolution M2 relies on.
- **G4 2D Graphics** — the renderer (objects/primitives/images/text + sprite-sheet animation) and the **paint trail**.
Tech & dependencies
 - **Tech: Web build:** A 2D renderer for bots, gadgets/weapons, projectiles, world map and text, with image-slicing sprite-sheet animation and a paint-trail renderer (a fading trail as the bot moves); SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** M2G4 (advanced VFX); every visible frame.
- **G5 Physics & Collision** — the combat collision (projectiles, hits, bot bodies).
Tech & dependencies
 - **Tech:** C++ + GLM math, projectile motion and collision/hit resolution against bots and the world map, damage application, and broadphase to keep cost stable with many projectiles.
 - **Prerequisites:** G1 (loop & tick), G2 (bot/weapon data).
 - **Feeds into:** The combat outcome/score; M2G13 (scoring).
- **G8 Enemy AI & Pathfinding** — the **FSM bot AI** whose behaviour is driven by stats + attached gadgets.
Tech & dependencies
 - **Tech:** A generic FSM (idle/seek/attack/retreat...) parameterised by the bot's stats and equipped gadgets, plus steering/pathfinding on the single world map, tuned from data (custom C++).
 - **Prerequisites:** G1 (tick, FSM), G2 (stat/gadget data).
 - **Feeds into:** M2G10 (advanced behaviours); the auto-battler's read.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**
Tech & dependencies
 - **Tech: Web build:** Tracy profiling of a full fight, ASan on MSVC, object pools for projectiles/particles, and validation on the mobile target (Pixel

9a and/or mobile web); the Firebase/Linux-server seam (accounts + build submission + match resolution + rating) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.

- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The mobile deliverable + a stable, deterministic sim for async PVP.

• **G2 ★ Content Editor — Core (Module Maker)** — author new bot attachments (gadgets/weapons) with their behaviours and stats.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: compose a new gadget/weapon from components, set its stats and behaviour, and preview it on a bot, with round-trip JSON serialization and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (the ECS/module data model), M1G4 (rendering for preview).
- **Feeds into:** All module content; the custom-tools showcase.

• **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android (or Emscripten) builds, tests (deterministic sim, collision, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

• **G8 Firebase — Accounts, Leaderboards & Analytics** — accounts, submitted bot builds, and the async-PVP **points rating / ranking**.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for submitted bot builds + match results + the rating/leaderboard (periodically updated), and

optional Analytics, via the REST API from C++; server-validated so ratings can't be forged.

- **Prerequisites:** G2 (build/result data model), M1 sim (deterministic resolution).
- **Feeds into:** The async-PVP + rating/ranking features; balancing telemetry.

- **G10 AI — Advanced** — the bot behaviours matured: stat/gadget-driven decision-making and difficulty/opponent variety.

Tech & dependencies

- **Tech:** Behaviour trees / richer FSM logic where equipped gadgets and stat thresholds change tactics (kite, rush, defend), plus opponent-bot variety for the auto-battler.
- **Prerequisites:** M1G8 (FSM AI foundation).
- **Feeds into:** Interesting fights across diverse builds; the "customisations have unique behaviours" pillar.

- **G13 Gameplay Systems II** — the **bot-build / stat / module economy + scoring** system.

Tech & dependencies

- **Tech:** C++ systems over the ECS for the point-allocation stat economy, the gadget/weapon effects (each a unique behaviour), and the timer/score win condition; balancing-data-driven.
- **Prerequisites:** M1G2 (modules/stats), M1G5 (combat → score).
- **Feeds into:** The build depth + the competitive rating at showcase; T2 module expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — a bot-builder auto-battler is a **systems-and-balance** game, so the design surface is large (the stat/point economy, the gadget/weapon behaviours, the FSM behaviour design, the rating/matchmaking and combat balance), fitting two designers; one artist can carry a **modular, combinatorial** bot look (chassis + interchangeable gadgets/weapons parts), the single world map, projectiles/paint-trail VFX and the builder UI (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). If the modular-part art volume outgrows one artist, trade the 2nd designer for a 2nd artist. *Scope note (advisory):* an FSM/ECS engine + combat sim + module maker + async-PVP + rating is a full but well-structured scope — the build-a-bot → deterministic auto-battle → score loop with a handful of gadgets and the module maker is the M1/M2 floor; the full async-PVP rating ladder and broad module variety are the stretch. Keep the combat sim **deterministic** so async matches resolve identically on the server.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the build-a-bot → auto-battle → score/rating loop and the stat/gadget behaviour model; primary benchmark e.g. Gladiabots / Super Auto Pets, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving build-a-bot → watch-it-fight → score is satisfying and readable) · prototype in Unity
- M1D3 User Research, Personas & Competitive Analysis (teardown of Gladiabots / Super Auto Pets / Robocraft; findings into the FDD)
- M1D5 Level & Progression Design (the rating/ranking progression and unlock pacing of gadgets/stats)
- M1D6 Systems & Economy Design (the point-allocation stat economy, the gadget/weapon behaviours and the scoring — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness

- M2D3 Systems Balance & Tuning — Data-Driven (stat/gadget balance and matchmaking/rating tuning via the team's JSON, no code changes)
- M2D5 Playtesting & Iteration Program (test build variety and fight readability, triage, re-test)
- M2D6 Progression & Retention Tuning (the rating ladder pacing and unlock cadence, kept fair and non-grindy)
- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the bot-builder (stat allocation + attachment), the combat HUD (timer/score) and the leaderboard.
 - M1U2 UX Flows & Wireframes — Menus → HUD — the build → submit → watch → rating flow wireframed end to end.
 - M1U3 Accessibility & Readability Foundations — legible stats/behaviour cues, contrast and touch targets on a phone.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — builder, combat, results and leaderboard navigable unaided on the mobile target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished builder + combat UX tested with players on the Pixel 9a.
 - M2U3 Menu & HUD Motion Design — build-confirm, combat-start, score and rating-update transitions animated so state reads clearly.
 - M2U4 Onboarding & Tutorial Design — teach build → attach → stats → watch → rate in the first session without a wall of text.
- **Art M1:** 1 artist (modular combinatorial bot parts + world + UI + VFX)
 - M1A1 ★ Art Bible / Proof of Concept (the visual direction — the "cleaning bot" tone, the modular-part language, readability of gadgets/weapons)
 - M1A2 Character Concepts & Model Sheets (the bot chassis + the interchangeable gadgets/weapons as a modular set)
 - M1A4 UI Art Concepts & Typography (the builder screen, stat allocation, combat HUD, leaderboard)

- **Art M2:**

- M2A1 ★ Critical Asset Production & In-Engine Visual Quality (the bot parts, world map, projectiles and UI implemented and consistent)
- M2A2 Character Animation — Extensive (bot movement, gadget/ weapon firing, hit/destroy reactions)
- M2A5 VFX — Critical Effects & Transitions (the paint trail, weapon/ gadget effects, projectile impacts and the destroy moment)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (workshop/ arena BGM, build-confirm, weapon-fire, hit, and destroy cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — the team's data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping combat SFX.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (BGM + build / weapon-fire / hit / gadget / score / destroy SFX, ≥6 types), scaling with combat intensity.
- M2TA2 ★ Audio Engine Functionality — audio properties de/ serialised and the full playback functionality the build needs (multi-channel mixing; distinct per-weapon/gadget cues).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the module maker), prototype demo, team post-mortem (wk14)